

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Направление: 02.03.02 «Фундаментальная информатика и информационные
технологии»
ООП: «Программирование и информационные технологии»

ОТЧЁТ О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

Тема задания: Вероятностный апостериорный анализ трудоёмкости
MSD-сортировки.

Выполнил: Ерхов Александр Александрович, группа 23.Б12–ПУ

Руководитель НИР: Никифоров Константин Аркадьевич, доцент кафедры
моделирования электромеханических и компьютерных
систем, кандидат физико-математических наук.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
Постановка задачи	4
1 Априорный анализ	5
2 Алгоритм	6
2.1 Генератор входных данных	6
2.2 Реализация сортировки	6
3 Математическое ожидание	9
3.1 Вычислительный эксперимент	9
3.2 Статистическая обработка	10
3.3 Анализ	11
4 Дисперсия	14
4.1 Статистическая обработка	14
4.2 Анализ	14
5 Гистограмма частот	15
6 Критерий согласия Пирсона	17
ЗАКЛЮЧЕНИЕ	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	20

ВВЕДЕНИЕ

Поразрядная сортировка не нуждается в обсуждении ее актуальности и применимости. Обсуждения стоит именно вопрос исследования трудоёмкости алгоритма. LSD-сортировка понятна и предсказуема, а вот MSD-сортировка – классический пример алгоритма класса NPR, как и, например, быстрая сортировка. И обычно вопрос её трудоёмкости заканчивается худшим случаем $O(nk)$, где n – количество элементов, k – количество разрядов элемента, и лучшим случаем $\Omega(n \log_b n)$, b – верхнее значение одного разряда (на каждом шаге по b корзин). Вопрос построения вероятностной модели, с последующим получением распределения с целью анализа среднего случая, является нетривиальным. Но в 2018 году ряд исследователей доказал центральную предельную теорему [1].

Цель – провести вероятностный апостериорный анализ трудоёмкости MSD-сортировки на основе результатов, полученных в [1]. Задачи:

1. Исследование материала [1].
2. Разработка алгоритма на основе модели, приведенной в [1].
3. Проведение вычислительного эксперимента.
4. Проверка вхождения математического ожидания и дисперсии в соответствующие доверительные интервалы.
5. Проверка центральной предельной теоремы с помощью критерия Пирсона.

Постановка задачи

Будем рассматривать словарь (массив слов). Пусть алфавит $\Sigma = \{0, 1\}$. Количество символов в слове непостоянно. Количество слов в словаре также заранее не известно – n . Необходимо отсортировать словарь в лексикографическом порядке.

Генерация словаря. Каждое слово генерируется независимо друг от друга. Начальный символ, в каждом слове, выбирается согласно начальному распределению μ , т. е. $P(\xi = 0) = \mu_0$, $P(\xi = 1) = \mu_1$, где $\mu_0 + \mu_1 = 1$. Далее используем марковские цепи: каждый следующий символ зависит только от предыдущего, в соответствии с матрицей переходов $P = [p_{ij}]$, $i, j \in \{0, 1\}$. При этом $p_{ij} \neq \frac{1}{2}, \forall i, j$ (Этот случай исключается как тривиальный). Также здесь введем понятие *энтропии*:

$$H = \sum_i \pi_i H_i, \text{ где } H_i = - \sum_j p_{ij} \log p_{ij}, \pi_0 = \frac{p_{10}}{p_{01} + p_{10}}, \pi_1 = \frac{p_{01}}{p_{01} + p_{10}}. \quad (1)$$

Причины. Алгоритм поразрядной MSD-сортировки находит применение в задачах, где определяющим фактором является значение старших разрядов ключей. В отличие от LSD-сортировки, обрабатывающей данные от младших разрядов к старшим и предполагающей фиксированную длину ключей, метод MSD обеспечивает раннее разделение элементов по наиболее значимым позициям, что особенно эффективно при работе с переменной длиной данных, таких как строки или идентификаторы. Он используется в системах, где требуется лексикографическое упорядочивание. Применение алгоритма оправдано в случаях, когда важна иерархическая структура ключей и наблюдается неоднородность их длины или распределения.

1 Априорный анализ

Как уже было описано выше, данный анализ проведен в [1]. Здесь приведены результаты, без доказательств.

B_n^μ – это случайная величина, представляющая количество операций с корзинами (*bucket operations*) в алгоритме сортировки (или внешнюю длину пути в соответствующем *trie*¹) при сортировке n строк, сгенерированных марковским источником с начальным распределением μ . При апостериорном анализе это будет означать количество операций сравнения.

Итого:

$$\mathbb{E}[B_n^\mu] = \frac{1}{H} n \log n + O(n); \quad (2)$$

$$\text{Var}(B_n^\mu) = \sigma^2 n \log n + O(n\sqrt{\log n}), \text{ где} \quad (3)$$

$$\sigma^2 = \frac{\pi_0 p_{00} p_{01}}{H^3} \left(\log(p_{00}/p_{01}) + \frac{H_1 - H_0}{p_{01} + p_{10}} \right)^2 + \frac{\pi_1 p_{10} p_{11}}{H^3} \left(\log(p_{10}/p_{11}) + \frac{H_1 - H_0}{p_{01} + p_{10}} \right)^2;$$
$$\frac{B_n^\mu - \mathbb{E}[B_n^\mu]}{\sqrt{\text{Var}(B_n^\mu)}} \xrightarrow{d} \mathcal{N}(0, 1). \quad (4)$$

¹Структура данных. Встречается как: Префиксное дерево, trie, prefix trie, ... [2].

2 Алгоритм

2.1 Генератор входных данных

Особенности были описаны в разделе *Постановка задачи*. Символы будут разыгрываться по методу Монте-Карло. Код:

```
1 import random
2
3 def generator(
4     n: int,
5     k: int,
6     delta: int,
7     mu: float,
8     P: list[float],
9 ):
10     array = []
11     for _ in range(n):
12         if random.random() < mu:
13             word = '0'
14         else:
15             word = '1'
16         real_num_k = k + random.randint(-delta, delta)
17         for __ in range(real_num_k - 1):
18             if word[-1] == '0':
19                 if random.random() < P[0]:
20                     word = word + '0'
21                 else:
22                     word = word + '1'
23             else:
24                 if random.random() < P[1]:
25                     word = word + '0'
26                 else:
27                     word = word + '1'
28         array.append(word)
29     return array
```

Пояснения к коду. μ — *float*, потому что, на самом деле второе число не нужно. Это важно при теоретическом анализе, но для применения Монте-Карло в данном контексте достаточно одного числа, так как алфавит с размерностью 2. Аналогично с матрицей P , которая выродилась в вектор. В итоге получаем массив с n словами, каждое из которых длиной $k \pm \Delta$, в соответствии с требованиями, описанными в разделе *Постановка задачи*.

2.2 Реализация сортировки

Был реализован *in-place* вариант MSD-сортировки, как классический:

```

1 def MSD_sort(
2     arr: list[str],
3     n: int
4 ) -> int:
5     stack = [(0, 0, n)] # digit, left, right
6     total_count = 0
7
8     while stack:
9         digit, left, right = stack.pop()
10        if right-left <= 1 or digit >= max(len(arr[i]) for i in range(left,
11        right)):
12            continue
13        count, groups = counting_sort(arr, left, right, digit)
14        total_count += count
15        for i in range(1, len(groups)):
16            new_left, new_right = groups[i]
17            if new_right - new_left > 1:
18                stack.append((digit+1, new_left, new_right))
19
20    return total_count

```

В процессе необходимо использовать устойчивую сортировку для сортировки отдельных разрядов. Была использована *counting sort*, так как она приемлема для сортировки символов и оптимальна из-за скудности размера теоретически возможного алфавита:

```

1 def counting_sort(
2     arr: list[str],
3     left: int,
4     right: int,
5     target_digit: int
6 ) -> tuple[int, list[tuple[int, int]]]:
7     n = right - left
8     R = 3 # None 0 1
9     C = [0] * (R + 1)
10    res = 0 # count of operations
11
12    for i in range(left, right):
13        if target_digit >= len(arr[i]):
14            ch = 0
15        elif arr[i][target_digit] == '0':
16            res += 1
17            ch = 1
18        else:
19            res += 1
20            ch = 2
21        C[ch+1] += 1
22

```

```

23     for i in range(R):
24         C[i+1] += C[i]
25
26     new_arr = [None] * n
27     for i in range(left, right):
28         if target_digit >= len(arr[i]):
29             ch = 0
30         elif arr[i][target_digit] == '0':
31             res += 1
32             ch = 1
33         else:
34             res += 1
35             ch = 2
36         new_arr[C[ch]] = arr[i]
37         C[ch] += 1
38
39     for i in range(n):
40         arr[i + left] = new_arr[i]
41
42     groups = []
43     start = left
44     for i in range(R):
45         end = C[i] + left
46         groups.append((start, end))
47         start = end
48
49     return res, groups

```

Все исходники выложены на GitHub [3].

3 Математическое ожидание

Начнем апостериорный анализ с проверки адекватности теоретического математического ожидания по отношению к экспериментальным данным.

3.1 Вычислительный эксперимент

Как уже было сказано выше, трудоёмкость алгоритма зависит не только от количества сортируемых данных, но и от распределения данных. Для начала зафиксируем распределение, ради проверки части связанной с n . Далее варьируем n . Идём по степеням двойки, чтобы за один вычислительный эксперимент проверить наиболее различные между собой n , т. е. вышло, что $n = 10, 20, 40, \dots, 40960$. Был достигнут порог лишь в 40960 элементов по одной лишь причине: необходимость в больших k (сравнимых с n или даже больше, был выбран величиной в $2n$). В априорном анализе авторы использовали бесконечное k , и это логично, ведь в противном случае ($k \ll n$) вычисление трудоёмкости тривиально, получается $O(nk)$. Мы же хотим по-максимуму понять зависимость от распределения. Получается примерно вот такой json:

```
1 {
2   "mu": 0.7,
3   "p": [
4     0.7,
5     0.2
6   ],
7   "experiments": [
8     {
9       "number": 60,
10      "n": 10,
11      "k": 20,
12      "delta": 10,
13      "experiments": [
14        140,
15        114,
16        ...,
17        110,
18        98
19      ]
20    },
21    {
22      "number": 60,
23      "n": 20,
24      "k": 40,
25      "delta": 20,
26      "experiments": [
27        268,
```

```

28     262,
29     ...
30     296
31 ]
32 },
33 ...
34 ]
35 }

```

number – это кол-во экспериментов для одного n .

3.2 Статистическая обработка

Для каждой выборки получаем эмпирическое среднее и дисперсию:

$$\bar{X} = \frac{1}{K} \sum_{i=1}^K X_i;$$

$$S^2 = \frac{1}{K-1} \sum_{i=1}^K (X_i - \bar{X})^2,$$

где K – количество экспериментов. В формулах фигурирует в таком виде, в коде – *number*

Далее идет построение доверительного интервала. Выбираем уровень значимости $\alpha = 0.05$. 95% – классический доверительный интервал. Был выбран Т-интервал (t-распределение Стьюдента), так как он оперирует эмпирической дисперсией, а в формуле дисперсии (3) присутствует O семантика, из-за чего применения Z-интервала не представляется возможным. Критическое значение для выбранного α составляет: $t_{crit} \approx 2.001$. Границы вычислительного интервала высчитываются как: $CI = [\bar{X} - t_{crit} \frac{S}{\sqrt{K}}, \bar{X} + t_{crit} \frac{S}{\sqrt{K}}]$. Также для каждой выборки был посчитан Н (формула (1)) и главная часть от мат. ожидания (формула (2)). Получился примерно вот такой json:

```

1 {
2   "mu": 0.7,
3   "p": [
4     0.7,
5     0.2
6   ],
7   "experiments": [
8     {
9       "number": 60,
10      "n": 10,
11      "k": 20,
12      "delta": 10,
13      "overline_X": 119.53333333333333,

```

```

14     "S^2": 559.7785310734462,
15     "alpha": 0.05,
16     "t_crit": 2.000995378088267,
17     "standard_error": 3.0544462975402658,
18     "CI": [
19         113.42140040933644,
20         125.64526625733022
21     ],
22     "H": 0.5445871749448702,
23     "main_part_E": 42.2812948767503
24 },
25 {
26     "number": 60,
27     "n": 20,
28     "k": 40,
29     "delta": 20,
30     "overline_X": 286.6,
31     "S^2": 1716.7864406779654,
32     "alpha": 0.05,
33     "t_crit": 2.000995378088267,
34     "standard_error": 5.349122109714149,
35     "CI": [
36         275.8964313816322,
37         297.3035686183678
38     ],
39     "H": 0.5445871749448702,
40     "main_part_E": 110.0184657803319
41 },
42 ...
43 ]
44 }

```

3.3 Анализ

Далее хотелось бы найти такое C , что $\frac{1}{H}n \log n + Cn \in CI, \forall n$ (формула (2)). Но такого C не существует. Для демонстрации проблемы обратимся к Рисунку 1. Пусть $\Delta = \bar{X} - \frac{1}{H}n \log n$. Тогда, по априорной оценке график $\frac{\Delta}{n}$ от n должен выглядеть как константа ($O(n)/n = O(1)$), но он выглядит как логарифм, значит константа при $n \log n$ неверная.

Найдем такое C , что $\frac{2}{H}n \log n + Cn \in CI$. Полученное число $C = 3.5608449917150553$, вычислялось через линейное программирование с помощью модуля *scipy*:

```

1 def find_C():
2     from scipy.optimize import linprog
3     with open("experiments/exp_E_update.json", "r") as f:

```

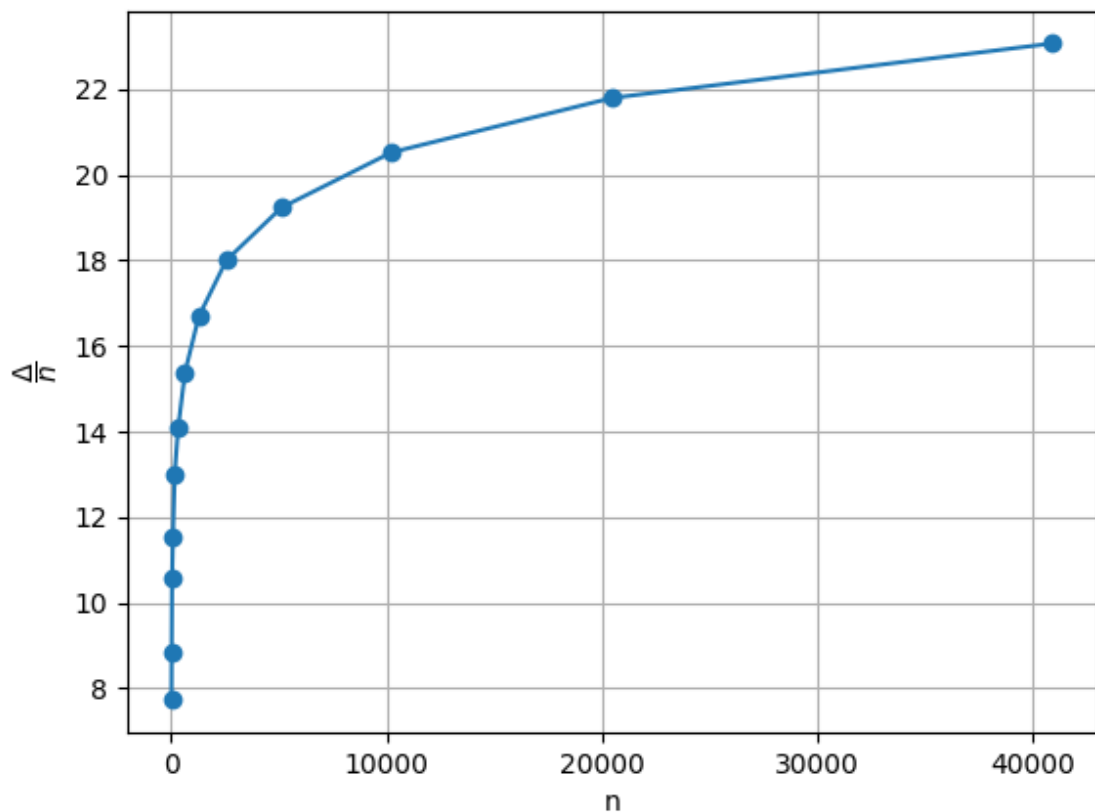


Рисунок 1 – График нормированных остатков

```

4     exp_E_update = json.load(f)
5
6     # Ax <= b
7     A = []
8     b = []
9
10    for p in exp_E_update["experiments"]:
11        n = p["n"]
12        H_val = p["H"]
13        ci_low, ci_high = p["CI"]
14        const_term = (2.0 / H_val) * n * log(n)
15
16        # C * n <= ci_high - const_term
17        A.append([n])
18        b.append(ci_high - const_term)
19
20        # B * n >= ci_low - A * (n*log(n)) => - B * n <= -ci_low + A * (n*
log(n))
21        A.append([-n])
22        b.append(const_term - ci_low)

```

```

23
24     res = linprog(c=[0], A_ub=A, b_ub=b, bounds=[(None, None)])
25
26     if res.success:
27         C_found = res.x[0]
28         print(f"C = {C_found}")
29         return C_found
30     else:
31         return None

```

Итого, можно считать, что представленное мат. ожидание является верным, хоть и с оговорками: неверная константа при $\frac{1}{H}n \log n$ (хотя это не так уж и важно).

4 Дисперсия

Необходимо также проверить и дисперсию. Используется те же выборки, что и с предыдущего раздела.

4.1 Статистическая обработка

Так как по результату (4) наша случайная величина имеет нормальное распределение, то для проверки дисперсии имеет место χ^2 -интервал:

$$\frac{(K-1)S^2}{\chi_{1-\alpha/2, K-1}^2} \leq Var \leq \frac{(K-1)S^2}{\chi_{\alpha/2, K-1}^2},$$

где

- $K - 1$ – число степеней свободы, кол-во экспериментов в одной выборке, ранее упоминалась также как *number*.
- $\chi_{1-\alpha/2, K-1}^2$ и $\chi_{\alpha/2, K-1}^2$ – квантили распределения Хи-квадрат с $K - 1$ степенями свободы на уровне значимости α (в нашем случае $\alpha = 0.05$). Вычислялось с помощью *scipy.stats.chi2*

Он был вычислен для каждой выборки.

4.2 Анализ

Далее необходимо найти такое C , что $\sigma^2 n \log n + C n \sqrt{\log n} \in CI, \forall n$ (формула (3)). И такое C нашлось! (в отличие от мат. ожидания): $C \in [32.622271995067116, 35.62588847912787]$. Для определенности, пусть $C = 34.124080237097495$ (середина). Вот используемый код:

```
1 def find_C_Var():
2     global_low = float("-inf")
3     global_high = float("inf")
4     with open("experiments/exp_E_update.json", "r") as f:
5         exp_E_update = json.load(f)
6     P = exp_E_update["P"]
7     for p in exp_E_update["experiments"]:
8         ci_low, ci_high = p["CI_chi_2"]
9         n = p["n"]
10        local_low = (ci_low - sigma_square(P)*n*log(n)) / (n * sqrt(log(n)))
11        local_high = (ci_high - sigma_square(P)*n*log(n)) / (n * sqrt(log(n)))
12        global_low = max(global_low, local_low)
13        global_high = min(global_high, local_high)
14    print(f'C = [{global_low}, {global_high}]')
```

Итого, теоретическую оценку дисперсии можно считать верной (и даже оценка при главном члене оказалась справедливой).

5 Гистограмма частот

Пусть $n = 10000$, так как для проверки нужно достаточно большое n (в [1] результаты (4) доказываются при $n \rightarrow \infty$). Тогда: $k = 2n = 20000$; $\Delta = n = 10000$; $\mu = 0.7$; $P = [0.7, 0.2]$. Поскольку эмпирическое среднее сходится к нормальному распределению, имеет место формула для нахождения необходимого количества экспериментов:

$$K = \left(\frac{z_{1-\alpha/2}\sigma}{\delta} \right)^2,$$

где

- δ – желаемая точность ($P(|\bar{X} - E| \leq \delta) = 1 - \alpha$ – необходимо, чтобы экспериментальное среднее отклонялось от истинного не более чем на величину δ с заданной вероятностью $1 - \alpha$). В настоящей работе $\delta = 0.05\sigma$.
- σ – стандартное отклонение трудоёмкости алгоритма ($\sqrt{Var}$).
- $z_{1-\alpha/2}$ – квантиль стандартного нормального распределения.

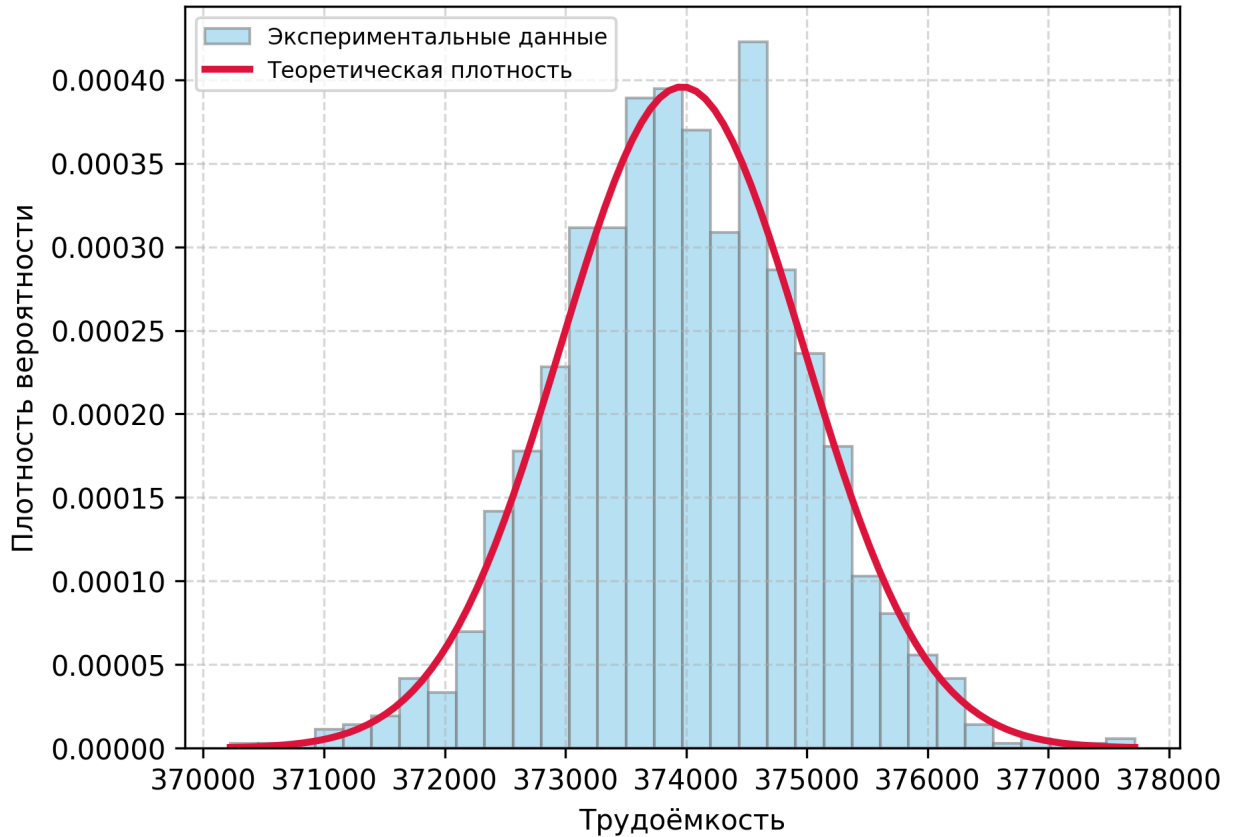


Рисунок 2 – Гистограмма частот

Полученная гистограмма частот – Рисунок 2 (количество бакетов автоматическое). Теоретическая плотность построена с помощью *scipy.stats.norm* с параметрами

$\mu = \overline{X}$ и $\sigma = S$. По графику, можно нестрого сказать, что утверждение (4) верно. Нет явных перекосов, асимметрии и несоответствий, кроме единичных незначительных выбросов.

6 Критерий согласия Пирсона

Необходимо проверить утверждение (4). Исходя из него и выбираем гипотезу H_0 – распределение B_n^μ сходится к нормальному.

Количество бакетов выбрано с помощью правила Стёрджеса:

$$b = 1 + \lfloor \log_2 K \rfloor,$$

где

- K – размер выборки.
- b – количество бакетов (корзин).

Далее идут упоминания особенностей реализации, связанных, главным образом, с *Python*. Вместо этого целесообразнее показать код (исходники можно найти в [3]):

```
1 def pirson():
2     with open("experiments/exp_for_frequency.json", "r") as f:
3         exp = json.load(f)
4         number = exp["number"]
5         experiments = exp["experiments"]
6         overline_X = sum(experiments) / number
7         s_sq = (sum([(p - overline_X)**2 for p in experiments])) / (number - 1)
8         alpha = 0.05
9
10        bins = 1 + floor(log2(number))
11        counts, bin_edges = np.histogram(experiments, bins = bins)
12        expected_edges = bin_edges.copy()
13        expected_edges[0] = -np.inf
14        expected_edges[-1] = np.inf
15
16        theory = np.diff(stats.norm.cdf(expected_edges, loc = overline_X, scale
17                                         = sqrt(s_sq)))
18        expected_cnt = theory * number
19
20        obs = list(counts)
21        exp = list(expected_cnt)
22        i = 0
23        while i < len(exp):
24            if exp[i] < 5 and len(exp) > 1:
25                if i < len(exp) - 1:
26                    exp[i+1] += exp[i]
27                    obs[i+1] += obs[i]
28                    exp.pop(i)
29                    obs.pop(i)
30            else:
```

```

30     exp[i-1] += exp[i]
31     obs[i-1] += obs[i]
32     exp.pop(i)
33     obs.pop(i)
34     i -= 1
35 else:
36     i += 1
37
38 counts_merged = np.array(obs)
39 expected_cnt_merged = np.array(exp)
40
41 assert len(expected_cnt_merged) > 3
42 assert np.all(expected_cnt_merged >= 5)
43
44 chi2_stat, p_value = stats.chisquare(f_obs=counts_merged, f_exp=
    expected_cnt_merged, ddof=2)
45
46 print(f"stat chi2: {chi2_stat}")
47 print(f"p-value: {p_value}")
48
49 if p_value > alpha:
50     print(f"p-value > {alpha}. Ok.")
51 else:
52     print(f"p-value <= {alpha}. Not ok.")

```

Упоминания здесь стоят 2 момента:

1. Строки 12-14. Это необходимо для того, чтобы далее в 16 строке правильно посчитались вероятности попадания в конкретный бакет. Иначе, если сразу передать *bin_edges*, не будут учитываться "хвосты" из-за чего итоговая сумма *theory* не будет дотягивать до 1.
2. Строки 22-36 – предобработка данных – объединение нерепрезентативных выборок. Иначе, есть шанс ошибки первого рода. Тем более, мы теряем всего 2 бакета.

Итого, критерий показал, что с уровнем значимости $\alpha = 0.05$ гипотеза H_0 является справедливой.

ЗАКЛЮЧЕНИЕ

В ходе выполнения научно-исследовательской работы был успешно проведен вероятностный апостериорный анализ трудоёмкости алгоритма поразрядной MSD-сортировки. На основе результатов априорного математического анализа была разработана программная модель, включающая генератор входных данных, который отвечал требованиям априорного анализа, и in-place реализацию алгоритма.

Вычислительный эксперимент и последующая статистическая обработка данных позволили сделать следующие выводы:

1. **Математическое ожидание:** Подтверждена теоретическая асимптотика $O(n \log n)$. Экспериментальные данные показали строгое соответствие функции $\frac{C}{H} n \log n + O(n)$. Вероятно, что экспериментальное полученное $C = 2$ (теоретически должно было получиться $C = 1$) связано главным образом с реализацией (в *counting_sort* мы дважды сравниваем символы). Опять же, асимптотика доказана, на константу можно внимание не обращать.
2. **Дисперсия:** Апостериорный анализ дисперсии доказал корректность теоретической оценки $Var(B_n^\mu)$. Построенные χ^2 -интервалы подтвердили сходимость эмпирической дисперсии к формуле (3).
3. **Распределение трудоёмкости:** Теорема о центральном предельном распределении трудоёмкости алгоритма полностью подтверждена на практике. Визуальный анализ гистограммы частот не выявил асимметрии или значимых отклонений от кривой нормального распределения. Статистическая проверка с использованием критерия согласия Пирсона подтвердила нулевую гипотезу о сходимости нормированной трудоёмкости к $\mathcal{N}(0, 1)$ на уровне значимости $\alpha = 0.05$.

Таким образом, поставленные в разделе *Введение* цель и задачи выполнены. Вероятностная модель, описывающая поведение MSD-сортировки на данных, сгенерированных марковским источником, доказала свою точность. Алгоритм, несмотря на высокую трудоёмкость в худшем случае, поддается строгому прогнозированию в среднем случае при известной матрице переходов алфавита.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] A Limit Theorem for Radix Sort and Tries with Markovian Input [Электронный ресурс] – Режим доступа: https://www.math.uni-frankfurt.de/neiningr/radix_sort.pdf – Дата обращения 21.04.2026

- [2] Префиксное дерево (trie) [Электронный ресурс] – Режим доступа: <https://habr.com/ru/companies/otus/articles/674378/> – Дата обращения 26.04.2026

- [3] SRW_MSD_1 Probabilistic a posteriori analysis of the complexity of MSD sorting. [Электронный ресурс] – Режим доступа: https://github.com/SashaErkhov/SRW_MSD_1 – Дата обращения 02.05.2026